

PSTAT100 Data Science Concepts and Analysis

Lab 1: Dataframe Overview with R

Claire Gan

Tuesday, January 13, 2026

R is a programming language widely used for statistical computing and data analysis. In this tutorial, we will focus on the `dplyr` package for data manipulation.

Objectives

This lab covers the following topics:

- **Dataframe basics**
 - Creating dataframes
 - Dataframe indexing and attributes
 - Adding, removing, and renaming variables
- **Operations on dataframes**
 - Slicing (selecting rows and columns)
 - Filtering (selecting rows that meet certain conditions)

If libraries are not downloaded in your machine, run the following command in your R console:
`install.packages("package_name")`

```
# Load libraries
library(dplyr)
library(ggplot2)
```

1. Creating DataFrames & Basic Manipulations

A **dataframe** in R is a table where each column has a specific type (numeric, character, etc.). It has two main indices:

- **Column index:** Labels for each column (usually strings).
- **Row index:** Ordinal numbers representing each row.

In this tutorial, we will cover:

1. Creating dataframes from scratch.
2. Retrieving dataframe attributes.
3. Indexing dataframes.
4. Adding, removing, and renaming columns.

We will use the example dataset: **fruit_info**, which contains information about different fruits.

1.1 Creating DataFrames from Scratch

In R, we use the `data.frame()` function to create a dataframe. Let's create a dataframe for the **fruit_info** dataset.

```
# Creating a dataframe using a list
fruit_info <- data.frame(
  Fruit = c("Apple", "Banana", "Cherry", "Date", "Elderberry"),
  Color = c("Red", "Yellow", "Red", "Brown", "Purple"),
  Weight = c(150, 120, 10, 25, 5)
)

# Display the dataframe
print(fruit_info)
```

	Fruit	Color	Weight
1	Apple	Red	150
2	Banana	Yellow	120
3	Cherry	Red	10
4	Date	Brown	25
5	Elderberry	Purple	5

1.2 Retrieving Attributes

Once the dataframe is created, you may want to check its attributes like the column names, the number of rows, and the number of columns.

```
# Checking the number of rows and columns  
nrow(fruit_info)
```

```
[1] 5
```

```
ncol(fruit_info)
```

```
[1] 3
```

```
# Displaying column names  
colnames(fruit_info)
```

```
[1] "Fruit" "Color" "Weight"
```

```
# Displaying the structure of the dataframe  
str(fruit_info)
```

```
'data.frame':  5 obs. of  3 variables:  
 $ Fruit : chr  "Apple" "Banana" "Cherry" "Date" ...  
 $ Color : chr  "Red" "Yellow" "Red" "Brown" ...  
 $ Weight: num  150 120 10 25 5
```

1.3 Indexing DataFrames

In R, you can access the data in a dataframe in various ways.

Column Indexing

You can access columns by name or by column index.

```
# Accessing columns by name index  
fruit_info$Fruit
```

```
[1] "Apple"      "Banana"      "Cherry"      "Date"      "Elderberry"
```

```
# Accessing columns by number index  
fruit_info[,1]
```

```
[1] "Apple"      "Banana"      "Cherry"      "Date"      "Elderberry"
```

Row Indexing

You can access rows by number or by condition.

```
# Accessing rows by number  
fruit_info[1, ]
```

```
      Fruit Color Weight  
1 Apple   Red    150
```

```
fruit_info[1:3, ]
```

```
      Fruit Color Weight  
1 Apple   Red    150  
2 Banana Yellow   120  
3 Cherry   Red     10
```

```
fruit_info[c(2,4,6), ]
```

```
      Fruit Color Weight  
2 Banana Yellow   120  
4 Date   Brown     25  
NA  <NA>  <NA>     NA
```

```
# Accessing rows by condition (Weight > 100)  
fruit_info[fruit_info$Weight > 100, ]
```

```
      Fruit Color Weight  
1 Apple   Red    150  
2 Banana Yellow   120
```

1.4 Adding, Removing, and Renaming Columns

Adding a Column

You can add a new column by assigning values to a new column name.

```
# Adding a new column for price (just an example)
fruit_info$Price <- c(1.2, 0.5, 2.0, 3.5, 4.0)

# View updated dataframe
print(fruit_info)
```

	Fruit	Color	Weight	Price
1	Apple	Red	150	1.2
2	Banana	Yellow	120	0.5
3	Cherry	Red	10	2.0
4	Date	Brown	25	3.5
5	Elderberry	Purple	5	4.0

Removing a Column

You can remove a column using `select()` from the `dplyr` package.

```
# Removing the Score column
library(dplyr)
fruit_info <- select(fruit_info, -Price)

# View updated dataframe
print(fruit_info)
```

	Fruit	Color	Weight
1	Apple	Red	150
2	Banana	Yellow	120
3	Cherry	Red	10
4	Date	Brown	25
5	Elderberry	Purple	5

Renaming Columns

Use `rename()` to change column names.

```
# Renaming columns
fruit_info <- rename(fruit_info, Fruit_Name = Fruit)

# View updated dataframe
print(fruit_info)
```

	Fruit_Name	Color	Weight
1	Apple	Red	150
2	Banana	Yellow	120
3	Cherry	Red	10
4	Date	Brown	25
5	Elderberry	Purple	5

Hand-on Exercise 1

(1) Adding a new column:

Using direct specification, add to the `fruit_info` table a new column called `rank1` containing integers 1, 2, 3, 4, and 5, which express your personal preference about the taste ordering for each fruit (1 is the tastiest; 5 is the least tasty). Make sure that the numbers utilized are unique - no ties are allowed.

Demo Codes:

```
# Adding a new column 'rank1' to express personal preference
fruit_info$rank1 <- c(2, 1, 3, 5, 4) # Ranking fruits based on preference

# View the updated dataframe
fruit_info
```

	Fruit_Name	Color	Weight	rank1
1	Apple	Red	150	2
2	Banana	Yellow	120	1
3	Cherry	Red	10	3
4	Date	Brown	25	5
5	Elderberry	Purple	5	4

(2) Creating a modified dataframe:

Now, create a new dataframe `fruit_info_mod1` with the same information as `fruit_info`, but with an additional column called `rank2`. We'll start by making a copy of the `fruit_info` dataframe to create the new dataframe `fruit_info_mod1`.

Demo Codes:

```
# Creating a new dataframe 'fruit_info_mod1' as a copy of 'fruit_info'
fruit_info_mod1 <- fruit_info
```

Then, using **name indexing**, add a column called `rank2` to the `fruit_info_mod1` dataframe that contains the same values in the same order as the `rank1` column.

Demo Codes:

```
# Adding 'rank2' to 'fruit_info_mod1' using the same values as 'rank1'
fruit_info_mod1$rank2 <- fruit_info_mod1$rank1

# View the updated dataframe
fruit_info_mod1
```

	Fruit_Name	Color	Weight	rank1	rank2
1	Apple	Red	150	2	2
2	Banana	Yellow	120	1	1
3	Cherry	Red	10	3	3
4	Date	Brown	25	5	5
5	Elderberry	Purple	5	4	4

When using the name indexing approach, the array assigned to the new variable must be **the same length** as the number of rows in the dataframe. What if we only assign values to certain rows?

(3) New column assignment with missing data:

For example, let's try adding a new column `rank3` and assign values only to the first two rows.

Demo Codes:

```
# Assign 'rank3' to only the first two rows
fruit_info_mod1[1:2, "rank3"] <- c(1, 2)

# View the updated dataframe with missing values in 'rank3'
fruit_info_mod1
```

	Fruit_Name	Color	Weight	rank1	rank2	rank3
1	Apple	Red	150	2	2	1
2	Banana	Yellow	120	1	1	2
3	Cherry	Red	10	3	3	NA
4	Date	Brown	25	5	5	NA
5	Elderberry	Purple	5	4	4	NA

Hints:

- **Partial Assignment:** We use row-based indexing (1:2) to assign values only to the first two rows of `rank3`. The remaining rows in `rank3` will have NA values by default because we didn't assign values to them.
- **Missing Values:** The unassigned rows will contain missing values (NA).

(4) Checking missing entry:

We can detect missing values using the `is.na()` method in R, which will return a logical dataframe indicating whether an entry is missing.

Demo Codes:

```
# Detect missing values in the dataframe
missing_data <- is.na(fruit_info_mod1)

# Print missing data frame
missing_data
```

	Fruit_Name	Color	Weight	rank1	rank2	rank3
[1,]	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
[2,]	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
[3,]	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
[4,]	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
[5,]	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE

Hints:

- **Missing Data Check:** The `is.na()` function checks for missing (NA) values in the dataframe. It returns a logical dataframe where `TRUE` indicates missing values.
- **Visualization:** This is helpful for identifying rows or columns with missing data.

It's often more helpful to see if any column has missing values. You can do this by appending `any()` to the `is.na()` function, which will return `TRUE` if there are any missing values in the column.

Demo Codes:

```
# Check if any column contains missing values
any_missing <- any(is.na(fruit_info_mod1))

# Check for missing data in each column
missing_columns <- colSums(is.na(fruit_info_mod1))

# Print the result
missing_columns
```

Fruit_Name	Color	Weight	rank1	rank2	rank3
0	0	0	0	0	3

Hints:

- **Missing Data Detection:** We use `colSums(is.na(fruit_info_mod1))` to check for missing data in each column. This will give us the number of missing entries per column.
- **Helpful for Clean-Up:** This helps identify which columns may need further attention or data cleaning.

(5) Removing column(s) with missing entry:

Once we've finished with some columns, we might want to remove them. In R, we can use `select()` from the `dplyr` package or direct indexing to remove columns.

Demo Codes:

```
# Removing a column using dplyr's select()
library(dplyr)
fruit_info_mod1 <- select(fruit_info_mod1, -rank3) # Remove 'rank3' column

# View the updated dataframe
fruit_info_mod1
```

	Fruit_Name	Color	Weight	rank1	rank2
1	Apple	Red	150	2	2
2	Banana	Yellow	120	1	1
3	Cherry	Red	10	3	3
4	Date	Brown	25	5	5
5	Elderberry	Purple	5	4	4

Hints:

- **Removing Columns:** We used the `select()` function from `dplyr` to remove the `rank3` column from the dataframe. The negative sign (`-`) indicates that the column should be removed.
- **Data Cleaning:** This is a common operation when you no longer need certain columns in the dataset.

2. Operations on Data Frames

With some basics in place, here you'll see how to perform subsetting operations on data frames that are useful for tidying up datasets.

- **Slicing:** Selecting columns or rows in chunks or by position.
- **Filtering:** Selecting rows that meet certain criteria.

We will illustrate these operations using a dataset comprising counts of baby names born in California each year from 1990 to 2018.

```
# Import baby names data
# (Put the csv file in the same folder as this .qmd file)
baby_names <- read.csv("baby_names.csv")

# Preview first few rows
head(baby_names)
```

	State	Sex	Year	Name	Count
1	CA	F	1990	Jessica	6635
2	CA	F	1990	Ashley	4537
3	CA	F	1990	Stephanie	4001
4	CA	F	1990	Amanda	3856
5	CA	F	1990	Jennifer	3611
6	CA	F	1990	Elizabeth	3170

Important note:

If you received some Error messages saying *Error in file(file, "rt") : cannot open the connection*, this usually means R cannot find your data file.

Make sure to put the `baby_names.csv` file in the **same folder** as this `.qmd` file.

Hand-on Exercise 2

(1) Checking dimensions:

Check the dimensions of the `baby_names` dataset and store them in `dimensions_baby_names`. In R, we use the `dim()` function to get the dimensions of a dataframe.

Demo Codes:

```
# Get the dimensions of the 'baby_names' dataframe
dimensions_baby_names <- dim(baby_names)

# Print the dimensions
dimensions_baby_names
```

```
[1] 190762      5
```

(2) Counting distinct years:

Count the number of occurrences of each distinct year in `baby_names`. Store the counts as `occur_per_year` (ordered by year), and the total number of years as `num_years`.

Demo Codes:

```
# Count occurrences of each year
occur_per_year <- table(baby_names$Year)

# Sort the occurrences by year
occur_per_year <- occur_per_year[order(as.numeric(names(occur_per_year)))]

# Get the number of years represented in the dataset
num_years <- length(unique(baby_names$Year))

occur_per_year
```

```
1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005
6261 6226 6304 6314 6241 6092 6036 5961 5976 6052 6284 6333 6414 6533 6708 6874
2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018
7075 7250 7158 7119 7010 6880 7007 6861 6952 6871 6770 6684 6516
```

```
num_years
```

```
[1] 29
```

Notes:

- **Sorting by Year:** We first count the occurrences of each year with the `table()` function, which gives us a named vector. The `order()` function is then used to sort the vector by the year, which is achieved by converting the names of the `occur_per_year` (which are stored as character strings) into numeric values with `as.numeric()`.
- **Number of Years:** The `unique()` function is used to get the distinct years, and `length()` gives the number of unique years.

2.1 Slicing: Selecting Rows and Columns

In this section, we'll cover two primary ways to slice a dataframe:

- Using **index names** to specify rows and columns.
- Using **integer positions** to specify rows and columns.

These methods are very useful for subsetting data and inspecting different portions of a dataframe.

1. Slicing with Index Names

To slice a dataframe by index names (column and row names), you can use the `[]` operator in R. You can select a specific entry by specifying both the row and column names.

```
# Selecting a single row and column by index name
baby_names[2, 'Name']
```

```
[1] "Ashley"
```

- **Single Index:** In this example, we select the second row and the `Name` column from the `baby_names` dataframe using row index 2 and column name `'Name'`.

2. Slicing with a List of Indices

You can also select multiple rows and specific columns by passing a **list** of indices or names.

```
# Selecting multiple rows and specific columns by index names
baby_names[c(2, 3), c('Name', 'Count')]
```

	Name	Count
2	Ashley	4537
3	Stephanie	4001

- **List of Indices:** We use `c(2, 3)` to select rows 2 and 3, and `c('Name', 'Count')` to select the columns `'Name'` and `'Count'`.

3. Slicing with Consecutive Indices

You can select a range of rows or columns using the colon `(:)` operator.

```
# Selecting rows 2 to 10 and columns from 'Year' to 'Count'
select(baby_names[2:10,], Year:Count)
```

	Year	Name	Count
2	1990	Ashley	4537
3	1990	Stephanie	4001
4	1990	Amanda	3856
5	1990	Jennifer	3611
6	1990	Elizabeth	3170
7	1990	Sarah	2843
8	1990	Brittany	2737
9	1990	Samantha	2720
10	1990	Michelle	2453

- **Consecutive Indices:** The `2:10` syntax selects rows 2 through 10, and `'Year': 'Count'` selects all columns from `Year` to `Count`.
- **`select(Year:Count)`:** This approach uses `dplyr`'s `select()` function to select columns starting from `'Year'` and ending at `'Count'`. It properly handles the selection of consecutive columns by name, ensuring that no errors occur.

4. Slicing with Integer Positions

In R, you can also slice data by integer positions using the `[,]` operator. To select a specific entry by position, you can specify row and column indices.

```
# Selecting a specific entry by position (row 2, column 3)
baby_names[2, 3]
```

```
[1] 1990
```

- **Single Position:** The `baby_names[2, 3]` selects the entry in the second row and third column by position.

5. Slicing with a List of Positions

You can select multiple rows and columns by specifying their integer positions.

```
# Selecting multiple rows and columns by position
baby_names[c(2, 3), c(3, 4)]
```

	Year	Name
2	1990	Ashley
3	1990	Stephanie

- **List of Positions:** The `c(2, 3)` selects the second and third rows, and `c(3, 4)` selects the third and fourth columns by position.

6. Slicing with Consecutive Positions

You can also slice consecutive rows or columns by specifying a range of positions.

```
# Selecting rows 2 through 11 and columns 2 through 5
baby_names[2:11, 2:5]
```

	Sex	Year	Name	Count
2	F	1990	Ashley	4537
3	F	1990	Stephanie	4001
4	F	1990	Amanda	3856
5	F	1990	Jennifer	3611
6	F	1990	Elizabeth	3170
7	F	1990	Sarah	2843
8	F	1990	Brittany	2737
9	F	1990	Samantha	2720
10	F	1990	Michelle	2453
11	F	1990	Melissa	2442

- **Consecutive Positions:** The 2:11 selects rows 2 through 11, and 2:5 selects columns 2 through 5 based on their integer positions.

2.2 Filtering

Filtering is the process of sifting out rows according to a criterion. In R, this can be accomplished using logical indexing, which creates a vector of TRUEs and FALSEs based on a comparison. Let's walk through a simple example:

Filter Names Based on Occurrence Count

Let's say we want to filter out all names with fewer than 1000 occurrences. First, we can define a logical vector that checks the `Count` column.

```
# Create a logical vector where TRUE means Count is greater than 1000
arr <- baby_names$Count > 1000

# View the logical array
head(arr, sum(arr)*0.05)
```

```

[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[13] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[25] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[37] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[121] FALSE FALSE FALSE FALSE FALSE

```

Once we have the logical array, we can apply it to filter the dataframe and select only the rows where Count is greater than 1000.

```

# Filter the dataframe using the logical array
baby_names_filtered <- baby_names[arr, ]

# View the filtered dataframe
head(baby_names_filtered)

```

	State	Sex	Year	Name	Count
1	CA	F	1990	Jessica	6635
2	CA	F	1990	Ashley	4537
3	CA	F	1990	Stephanie	4001
4	CA	F	1990	Amanda	3856
5	CA	F	1990	Jennifer	3611
6	CA	F	1990	Elizabeth	3170

Checking Dimensions Before and After Filtering

To get a sense of how many rows were filtered, we can compare the dimensions of the original and filtered dataframes.

```

# Compare dimensions before and after filtering
print(dim(baby_names))

```

```

[1] 190762      5

```



```
print(dim(baby_names_filtered))
```

```
[1] 2517    5
```

Notice that the filtered dataframe is much smaller than the overall dataframe – only about **2000** rows correspond to a name occurring more than 1000 times in a year for a gender.

Common Comparison Operators

Here are some commonly used comparison operators in R for filtering data:

Symbol	Usage	Meaning
==	a == b	Does a equal b?
<=	a <= b	Is a less than or equal to b?
>=	a >= b	Is a greater than or equal to b?
<	a < b	Is a less than b?
>	a > b	Is a greater than b?
!	!p	Negates p (logical NOT)
&	p & q	p AND q
	p q	p OR q

What if instead you wanted to filter using multiple conditions? Here's an example of retrieving rows with counts exceeding 1000 for only the year 2001:

Filtering with Multiple Conditions

You can combine multiple conditions to filter the dataframe in more complex ways. For example, let's retrieve all names where **Count** exceeds 1000 and the **Year** is 2001:

```
# Filter using two conditions
filtered_data <- baby_names[(baby_names$Year == 2001) & (baby_names$Count > 1000), ]

# View the filtered data
head(filtered_data, dim(filtered_data)[1]*0.1)
```

```
      State Sex Year   Name Count
40184    CA   F 2001   Emily 2928
40185    CA   F 2001  Ashley 2715
```

40186	CA	F	2001	Samantha	2535
40187	CA	F	2001	Jessica	2244
40188	CA	F	2001	Alyssa	2059
40189	CA	F	2001	Jennifer	2026
40190	CA	F	2001	Natalie	1724
40191	CA	F	2001	Elizabeth	1704
40192	CA	F	2001	Alexis	1700

- **Combining Conditions:** In this example, we combine two conditions using the & operator. We filter for rows where Year equals 2001 and Count is greater than 1000.

Hand-on Exercise 3:

In R, you can also slice rows based on specific conditions by using logical indexing. For example, you can select rows where a certain column satisfies a condition.

(1.) Selecting Rows with Count Greater than 100

Slice rows where 'Count' is greater than 100 from the baby_name dataframe

Demo Codes:

```
# Slice rows where 'Count' is greater than 100
head(baby_names[baby_names$Count > 100, ])
```

	State	Sex	Year	Name	Count
1	CA	F	1990	Jessica	6635
2	CA	F	1990	Ashley	4537
3	CA	F	1990	Stephanie	4001
4	CA	F	1990	Amanda	3856
5	CA	F	1990	Jennifer	3611
6	CA	F	1990	Elizabeth	3170

(2.) Slicing Rows Based on Column Value Range

Selecting Rows for Years 2000 to 2005 from the baby_name dataframe.

Demo Codes:

```
# Slice rows where the 'Year' is between 2000 and 2005
head(baby_names[baby_names$Year >= 2000 & baby_names$Year <= 2005, ])
```

	State	Sex	Year	Name	Count
36417	CA	F	2000	Emily	2958
36418	CA	F	2000	Ashley	2831
36419	CA	F	2000	Samantha	2579
36420	CA	F	2000	Jessica	2484
36421	CA	F	2000	Jennifer	2263
36422	CA	F	2000	Alyssa	2005

- **Range Condition:** Here, we use a logical condition to select rows where the **Year** is between 2000 and 2005 (inclusive). The logical condition is used within the indexing brackets to filter rows.